VASP CONTCAR Trajectory Reader Specification

Native-Velocity Parsing for Watcher-Generated Concatenated MD Restart Records

mdstats

2026-07-15 (CT0-CT3 implementation revision)

Contents

1 Purpose and status

This document specifies the implemented custom VASP trajectory reader

`mdstats/io/vasp_contcar_trajectory.py`

and its public dispatch through

```
mdstats.read_vasp_frames(..., format="vasp-contcar-trajectory")
```

The format is intended for molecular-dynamics runs in which native per-frame ionic velocities are required for VACF, velocity-spectrum, VDOS, and related statistics, but VASP is not writing a convenient native trajectory container with velocities.

The central invariant is

> every accepted frame contains the native Cartesian CONTCAR velocity block.

A missing, malformed, truncated, or unsupported velocity block is a hard parse error. The reader never substitutes a finite-difference estimate from positions.

2 Format provenance and naming

2.1 VASP-defined record format

A VASP `CONTCAR` is compatible with the POSCAR grammar. For MD runs, VASP identifies three logical contents: structure, ionic velocities, and predictor–corrector restart state [1,2]. The following rules are borrowed from the official VASP documentation:

- one or three lattice scaling values;
- row-wise lattice vectors;
- named species and per-species counts;
- optional selective-dynamics flags;
- direct or Cartesian positions;
- optional lattice-velocity block;
- a VASP-written blank Cartesian ionic-velocity mode line;
- ionic velocities in Å/fs;
- predictor–corrector restart preamble after the velocity block.

2.2 mdstats-defined stream format

`TRAJECTORY` is **not a native VASP output filename or official VASP trajectory format**. It is an mdstats workflow format defined as

> a chronological byte-stream concatenation of complete VASP MD `CONTCAR` restart records, with no extra inter-record delimiter.

The framing, explicit source-format selector, native-velocity requirement, three-array predictor grammar for this observed VASP variant, provenance schema, and error policy are mdstats designs.

The canonical format identifier is

`vasp-contcar-trajectory`

3 Reproducible creation workflow

The package supplies two executable examples:

```
examples/vasp_contcar_trajectory/watch_contcar.sh
examples/vasp_contcar_trajectory/vasp_contcar.sh
```

The first script polls the live VASP `CONTCAR`, waits for a stable file size and modification signature, copies the candidate, verifies that the source did not change during the copy, rejects byte-identical duplicates, and writes files such as

```
velocity_frames/CONTCAR.00000001
velocity_frames/CONTCAR.00000002
...
velocity_frames/CONTCAR.00001234
```

The second script prevents a stale pre-run `CONTCAR` from being archived as the first frame, starts the watcher in the background, invokes VASP, and terminates the watcher after a final grace period.

3.1 Preparation

Copy or invoke the scripts from the VASP working directory, edit the VASP launch line for the local executable and scheduler, and run

```
chmod +x watch_contcar.sh vasp_contcar.sh
./vasp_contcar.sh
```

The supplied caller contains the placeholder

```
srun /path/to/vasp_std
```

which must be replaced with the site's actual launch command.

3.2 Concatenation recipe

The watcher writes eight-digit zero-padded suffixes. Therefore lexical shell glob order is chronological capture order. From the snapshot directory, create the custom file with the exact recipe

```
cd velocity_frames
cat CONTCAR.* > TRAJECTORY
```

Equivalently, from the VASP working directory:

```
cat velocity_frames/CONTCAR.* > TRAJECTORY
```

The literal command

```
cat CONTCAR.* > TRAJECTORY
```

is normative when executed inside the directory that contains only the zero-padded snapshot files.

Do not include `CONTCAR.before_md`, temporary copies, logs, or unrelated files. The zero padding is part of the reproducibility contract; names such as `CONTCAR.1`, `CONTCAR.2`, and `CONTCAR.10` do not sort chronologically under a plain shell glob.

3.3 Watcher limitations

The suffix generated by the example watcher is a captured-snapshot sequence. It equals the VASP ionic-step number only if every `CONTCAR` update is observed and saved. Polling cannot prove that no write was missed when VASP updates the file faster than the watcher can detect stable states.

Consequently:

- the concatenated stream preserves observed chronological order;
- the stream does not independently encode original VASP ionic-step labels;

- `timestep_fs` supplied to the reader is authoritative;
- the user must choose a polling interval suitable for the run and filesystem;
- a slow parallel filesystem may require a longer stability interval.

4 Supported record grammar

For the implemented first version, one record has the following ordered grammar:

```
comment                                    1 line
scale                                      1 line
lattice vectors                            3 lines
species names                              1 line, mandatory
species counts                             1 line
[Selective dynamics]                       optional 1 line
position mode                              1 line
positions                                  N lines
[Lattice velocities and vectors block]     optional 8 lines total
ionic velocity mode                        1 line, blank or Cartesian
ionic velocities                           N lines
blank predictor separator                  1 line
predictor initialization state             1 line
embedded POTIM                             1 line
four-value thermostat restart state        1 line
predictor arrays                           3 N lines
```

No additional delimiter is inserted before the next record. The next byte after the final predictor vector begins the next comment line.

The reader deliberately supports the complete predictor layout observed in the supplied MD files: three $N \times 3$ arrays. The VASP Wiki describes the predictor section but labels its total line count as variable [1]. If a future VASP version writes another predictor layout, this reader fails rather than guessing the record boundary.

5 Public API

```python
from collections.abc import Mapping
from pathlib import Path
from typing import Literal


def read_vasp_frames(
    filename: str,
    *,
    format: Literal[
        "vasp-xml",
        "vasp-xdatcar",
        "vasp-contcar-trajectory",
    ] | None = None,
    start: int | None = None,
    stop: int | None = None,
    stride: int = 1,
    timestep_fs: float | None = None,
    reconstruct_velocities: bool = True,
    frame_semantics: FrameSemantics | str = FrameSemantics.TRAJECTORY,
    strict: bool = True,
    mass_map: Mapping[str, float] | None = None,
) -> AtomisticFrameCollection:
    ...
```

Example:

```
from mdstats import read_vasp_frames

trajectory = read_vasp_frames(
    "TRAJECTORY",
    format="vasp-contcar-trajectory",
    timestep_fs=1.0,
)
```

5.1 Explicit format selection

The custom format must be selected explicitly. A file named `TRAJECTORY` has no native VASP suffix or magic header that distinguishes it from a single POSCAR or CONTCAR record.

Automatic detection remains limited to `.xml` and basenames ending in `XDATCAR`.

5.2 Time semantics

`timestep_fs` is mandatory and means

> physical time between adjacent archived CONTCAR records in the concatenated stream.

It is not necessarily equal to VASP `POTIM`. If the watcher archives every k-th ionic step,

$$\Delta t_{\mathrm{saved}} = k\,\mathrm{POTIM}.$$

For source record index j,

$$t_j = j\,\Delta t_{\mathrm{saved}}.$$

The first record in the complete stream has relative time zero. A selected subtrajectory preserves the original source-record time; `start=10` does not reset the first retained frame to zero.

`steps` is set to `None` because the concatenated bytes no longer retain the original watcher filenames or independently verified VASP ionic-step labels. Selected source record indices are retained in metadata.

5.3 Selection

- `start` and `stop` are nonnegative source-record indices or `None`;
- `stride` is a positive integer;
- selection follows Python half-open slice semantics;
- the parser still validates every record in the stream so source count, fixed-species consistency, and truncation are checked globally;
- an empty selection is an error.

5.4 Frame semantics

The format is intrinsically time ordered. `frame_semantics` must be `trajectory`. Reading it as an independent ensemble is rejected because that would discard the native velocity field in the normalization pipeline.

5.5 Mass policy

CONTCAR does not embed POTCAR `POMASS` data. Masses are resolved in this order:

1. user values from `mass_map` for matching element symbols;
2. ASE standard atomic masses for remaining species.

Every explicit mass must be finite, positive, and expressed in atomic mass units.

6 Numerical interpretation

6.1 Lattice scaling

For one positive scale s and raw row-vector lattice H_0,

$$H = sH_0.$$

For one negative value $-V_*$, the value denotes a requested positive cell volume [1]. The uniform scale is

$$s = \left(\frac{V_*}{|\det H_0|} \right)^{1/3}.$$

For three positive values (s_x, s_y, s_z), the Cartesian components are scaled columnwise:

$$H = H_0 \operatorname{diag}(s_x, s_y, s_z).$$

Cartesian positions receive the same Cartesian scaling. Direct positions are already fractional.

All positions are converted to fractional row vectors before entering the source-independent normalizer:

$$\mathbf{r}_i = \mathbf{s}_i H.$$

6.2 Periodic positions

VASP notes that MD positions in CONTCAR may or may not be rebased into the unit cell depending on `MDALGO` [2]. Therefore the source parser does not require fractional components to lie in $[0, 1)$.

The existing mdstats trajectory normalizer:

1. wraps periodic components for image comparison;
2. computes minimum-image fractional increments;
3. accumulates a continuous fractional trajectory.

The position unwrapping is independent of the native velocity field.

6.3 Native ionic velocities

VASP states that the velocity section written to CONTCAR begins with an empty mode line, is Cartesian, receives no POSCAR scale-factor multiplication, and is written in Å/fs [1]. The reader also accepts an explicit Cartesian mode line beginning with `C`, `c`, `K`, or `k`.

The exact internal conversion is

$$1 \text{ Å/fs} = 1000 \text{ Å/ps},$$

so

$$\mathbf{v}_i^{\text{mdstats}} = 1000\, \mathbf{v}_i^{\text{CONTCAR}}.$$

Direct-mode velocity records are rejected in the first version. No position scale is applied to Cartesian velocities.

`reconstruct_velocities` does not weaken this rule. For the custom format, the normalizer is always called with

```
reconstruct_missing_velocities=False
```

and successful provenance must contain

```
collection.provenance.velocity_source == "native"
```

6.4 Lattice velocities

When a record begins the optional section with

`Lattice velocities and vectors`

this reader consumes and validates:

- one initialization-state line;
- three lattice-velocity vectors;
- three repeated lattice vectors.

The current `AtomisticFrameCollection` has no cell-velocity field. Therefore the values are not exposed as an analysis observable; presence is retained in metadata.

6.5 Predictor–corrector restart state

The reader consumes the predictor section to verify record completeness and locate the next record. It retains only aggregate metadata:

- initialization state;
- embedded `POTIM`;
- predictor-array count;
- predictor section presence.

The three arrays are not labelled as forces, accelerations, or independent positions. Their detailed internal meaning is not inferred beyond the VASP restart documentation.

7 Internal data structures

```
@dataclass(slots=True)
class _ContcarMDRecord:
    comment: str
    symbols: tuple[str, ...]
    counts: NDArray[np.int64]
    cell: NDArray[np.float64]
    fractional_positions: NDArray[np.float64]
    velocities_angstrom_per_ps: NDArray[np.float64]
    coordinate_mode: str
    selective_dynamics: bool
    lattice_velocity_block_present: bool
    predictor_initialization_state: int
    embedded_potim_fs: float
    predictor_array_count: int
```

The line reader tracks the physical source line number. Each record is complete and validated before it is yielded.

8 Parsing algorithm

```
validate explicit saved-frame timestep and slice
open plain/gzip/bzip2/xz text stream
record_index <- 0
for each record until EOF:
    read comment
    parse scale and three lattice rows
    parse named species and counts
    parse optional selective-dynamics marker
    parse N positions
    parse optional lattice-velocity section
```

```
    require blank or Cartesian velocity mode
    parse N finite native velocity vectors
    convert A/fs -> A/ps
    require predictor separator and preamble
    consume exactly three N x 3 predictor arrays
    compare species/counts with first record
    retain record if selected
    record_index <- record_index + 1
validate embedded POTIM consistency
resolve masses
stack selected arrays into RawFrameCollection
normalize with velocity reconstruction disabled
return AtomisticFrameCollection
```

For T records and N atoms, parsing cost is

$$O(TN)$$

because each record contains $5N$ atom-vector lines in the implemented grammar: N positions, N velocities, and $3N$ predictor vectors.

The retained numerical arrays require $O(T_s N)$ memory for T_s selected frames. Unselected records are validated and released immediately.

9 Raw and normalized mapping

The source parser constructs

```
RawFrameCollection(
    atomic_numbers=(T, N),
    masses=(T, N),
    frame_ids=arange(T),
    steps=None,
    times=source_record_indices * timestep_fs * 1e-3,
    cells=(T, 3, 3),
    origins=zeros((T, 3)),
    pbc=(True, True, True),
    coordinate_kind="wrapped_fractional",
    coordinates=(T, N, 3),
    velocities=(T, N, 3),
    forces=None,
    stresses=None,
    energies=None,
)
```

The final collection stores:

- time in ps;
- cells and Cartesian positions in Å;
- native Cartesian velocities in Å/ps;
- masses in amu;
- time-unwrapped fractional positions;
- no force, stress, energy, pressure, or temperature fields.

10 Provenance and metadata

Required provenance:

```
FrameCollectionProvenance(
    source_format="vasp-contcar-trajectory",
    velocity_source="native",
    coordinate_normalization="minimum_image_inferred",
    stress_source=None,
    units_source=(
        "VASP CONTCAR: Angstrom, Angstrom/fs converted to Angstrom/ps, fs"
    ),
)
```

Required metadata includes:

```
vasp_input_format
custom_format
creation_workflow
source_frame_count
selected_frame_count
selected_source_record_indices
saved_frame_timestep_fs
time_source
time_origin
source_step_labels_available
velocity_block_required
velocity_units_in_file
velocity_conversion_to_internal
predictor_corrector_present
predictor_array_count
embedded_potim_fs
embedded_potim_constant
implied_save_stride
lattice_velocity_block_present
selective_dynamics_present
position_coordinate_modes
mass_source
species_names
species_counts
```

10.1 Embedded POTIM diagnostic

The explicit `timestep_fs` defines the time axis. The embedded predictor-block `POTIM` is diagnostic.

- nonfinite or nonpositive embedded values are errors;
- changing embedded `POTIM` is an error in strict mode and a warning otherwise;
- if

$$q = \frac{\Delta t_{\text{saved}}}{\text{POTIM}}$$

  is close to a positive integer, `implied_save_stride = round(q)`;
- otherwise a warning is emitted and the implied stride is unavailable.

A matching ratio does not prove that the watcher captured every update.

11 Errors and diagnostics

```
class VaspContcarTrajectoryError(FrameCollectionError): ...
class TruncatedVaspRecordError(VaspContcarTrajectoryError): ...
class MissingNativeVelocityError(VaspContcarTrajectoryError): ...
class InconsistentVaspRecordError(VaspContcarTrajectoryError): ...
```

Every structural parse error reports:

- source path;
- one-based record number;
- one-based source line number;
- expected section;
- failure description.

Representative failure:

```
TRAJECTORY record 817, line 696914:
ionic velocity 164 of 168: reached end of file before the record was complete.
```

The parser detects partial watcher copies through truncated position, velocity, or predictor sections.

12 Supported variants

The first implementation supports:

- fixed or frame-dependent cells;
- direct or Cartesian positions;
- one positive, one negative-volume, or three positive scale values;
- optional selective dynamics;
- optional lattice velocities;
- blank or explicit Cartesian ionic-velocity mode;
- named VASP-5/6 species lines;
- plain, gzip, bzip2, and xz input;
- explicit element mass overrides;
- nonnegative `start` and `stop` with positive stride.

13 First-version non-goals

The reader does not recover or infer:

- energies;
- forces;
- stress or pressure;
- temperature;
- POTCAR masses without `mass_map`;
- original snapshot filenames after concatenation;
- missing ionic-step labels;
- watcher frame-loss detection;
- physical predictor-array interpretation;
- direct-mode ionic velocity conversion;
- VASP-4 counts-only species;
- optional or differently sized predictor layouts;
- relaxation CONTCAR collections lacking complete MD restart blocks.

14 Validation requirements

Synthetic focused tests must verify:

1. two records parse without an added delimiter;
2. native Cartesian velocities are preserved after the exact factor of 1000;
3. missing or direct-mode velocity blocks fail even when general reconstruction is enabled;
4. source provenance is `native`;
5. periodic positions unwrap across a boundary;
6. explicit saved-frame timing and slicing are correct;

7. selective-dynamics flags are consumed;
8. Cartesian position scaling follows one/three-value POSCAR rules;
9. optional lattice-velocity sections do not shift the ion-velocity boundary;
10. species/count changes fail;
11. truncated predictor arrays report record and line context;
12. embedded POTIM changes obey strict/warning policy;
13. mass overrides are honored;
14. compressed input is equivalent;
15. parsed collections are accepted by `compute_vacf()` and `compute_velocity_spectrum()` with native-velocity metadata.

Real-data acceptance must verify:

- `CONTCAR.00000001` parses as one 168-atom frame;
- `TRAJECTORY` parses as 1500 complete frames;
- the first record agrees between the standalone and concatenated files;
- all velocities are finite and retain exact text values after unit conversion;
- all records retain the same species/count ordering;
- downstream VACF and Welch spectrum functions consume the result without finite-difference reconstruction.

15 Acceptance criterion

The reader is accepted when

> the watcher-generated `TRAJECTORY`, read with an explicit saved-frame timestep, produces a time-ordered `AtomisticFrameCollection` containing every complete archived record, consistent atom identity, frame-dependent cells and unwrapped positions, and native Cartesian velocities converted exactly from Å/fs to Å/ps, with no finite-difference velocity path available.

16 References

[1] VASP Software GmbH, **POSCAR**, VASP Wiki. Full format specification, including scale factors, position modes, lattice velocities, ionic velocities, and predictor–corrector restart data. https://vasp.at/wiki/POSCAR, accessed 2026-07-15.

[2] VASP Software GmbH, **CONTCAR**, VASP Wiki. MD structure, velocity, and predictor–corrector block description. https://vasp.at/wiki/CONTCAR, accessed 2026-07-15.

[3] A. H. Larsen et al., "The Atomic Simulation Environment - A Python library for working with atoms," *Journal of Physics: Condensed Matter* **29**, 273002 (2017). DOI: `10.1088/1361-648X/aa680e`. ASE standard atomic masses are the fallback when no explicit `mass_map` entry is supplied.